

TASK 13: #51626

GOAL: To move/sync the backups taken in server to the S3 bucket.

1. create an IAM Role which allows access to S3, and attach with your aws instance.
2. login to the console[CLI] of the server and configure a mariadb server.
3. write a shell script for taking up the backups of databases to a directory.
4. configure aws-cli within the server and synchronize the backup directory with any of the aws s3 buckets.
5. verify the backups in S3.

Step 1: IAM Role (AWS Console)

Done in the console, no commands needed.

- IAM > Roles > Create role > AWS service > EC2.
- Attach the policy AmazonS3FullAccess (or a policy limited to your bucket).
- EC2 > select instance > Actions > Security > Modify IAM role > choose the role.

Why: the server gets S3 access automatically, so no access keys are stored on it.

Step 2: Log in and install MariaDB

```
ssh -i your-key.pem ec2-user@<EC2-PUBLIC-IP>
```

Connects to the server from your computer using your key file.

```
sudo dnf install -y mariadb-server
```

Installs the MariaDB server (Amazon Linux 2023). On other systems: mariadb-server via dnf (RHEL) or apt (Ubuntu). If the name is not found, run: dnf search mariadb | grep server

```
sudo systemctl enable --now mariadb
sudo systemctl status mariadb
sudo mysql_secure_installation
```

- enable --now: starts MariaDB now and on every reboot.
- status: should show active (running).
- mysql_secure_installation: sets the root password and removes insecure defaults.

Step 3: Create test data

```
sudo mysql -u root -p

CREATE DATABASE company;
USE company;
CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50));
```

```
INSERT INTO employees VALUES (1,'Anu'),(2,'Ravi');
EXIT;
```

Logs in to MariaDB and creates a sample database with two rows, so there is something to back up.

Step 4: Save the MariaDB login for the script

```
sudo vi /root/.my.cnf
[client]
user=root
password=YourRootPassword

sudo chmod 600 /root/.my.cnf
```

Why: the backup script can log in without typing a password (needed for cron).

Step 5: Backup script

```
sudo mkdir -p /backup/mariadb
sudo nano /usr/local/bin/db_backup.sh
```

mkdir -p: p means parents. It creates /backup and /backup/mariadb together and does not fail if they exist. nano opens the editor to write the script.

Script contents:

```
#!/bin/bash
BACKUP_DIR="/backup/mariadb"
DATE=$(date +%F_%H-%M)
RETENTION_DAYS=7
mkdir -p "$BACKUP_DIR"

DATABASES=$(mysql -N -e "SHOW DATABASES;" | grep -Ev
"^(information_schema|performance_schema|mysql|sys)$")

for DB in $DATABASES; do
    mysqldump --single-transaction --routines --triggers "$DB" | gzip >
"$BACKUP_DIR/${DB}_${DATE}.sql.gz"
    echo "Backup of $DB completed"
done

find "$BACKUP_DIR" -type f -name "*.sql.gz" -mtime +$RETENTION_DAYS -delete
```

- Lists all user databases (skips system ones).
- mysqldump exports each database, gzip compresses it, and the date is added to the file name.
- find ... -delete removes local backups older than 7 days.

Make it executable, run it, and check the result:

```
sudo chmod +x /usr/local/bin/db_backup.sh
sudo /usr/local/bin/db_backup.sh
ls -lh /backup/mariadb
```

Step 6: Install and check AWS CLI

```
cd /tmp
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

Downloads, unzips and installs AWS CLI v2 (fixes 'aws: command not found'). For ARM servers use the aarch64 zip file. No aws configure is needed because the IAM role supplies credentials.

```
aws sts get-caller-identity
aws configure list
aws s3 ls
```

- sts get-caller-identity: shows which identity (the IAM role) the server is using.
- configure list: TYPE should say iam-role, and region is detected automatically.
- s3 ls: lists your buckets, proving S3 access works.

Step 7: Create the S3 bucket (AWS Console)

- S3 > Create bucket, enter a globally unique name, choose the same region as the EC2 instance (ap-south-1).
- Keep Block all public access ticked and default encryption on.

Your bucket: backupbucket-176938214725-ap-south-1-an

Step 8: Sync the backups to S3

```
aws s3 sync /backup/mariadb s3://backupbucket-176938214725-ap-south-1-an/mariadb/
```

Part	Meaning
aws s3 sync	Copies new or changed files from the source to the destination.
/backup/mariadb	Source: the local backup folder.
s3://.../mariadb/	Destination: the mariadb/ prefix (folder) in the bucket.

Successful output looks like: upload: ../../backup/mariadb/company_2026-09-21_05-03.sql.gz to s3://...

Tip: the URL must be one unbroken string with no spaces. A space after s3:// caused the earlier 'Unknown options' error.

Step 9: Verify the backup in S3

```
aws s3 ls s3://backupbucket-176938214725-ap-south-1-an/mariadb/ --human-readable
aws s3 ls s3://backupbucket-176938214725-ap-south-1-an/ --recursive --human-readable
```

Lists the files in the bucket. In the console: S3 > your bucket > Objects > mariadb/ and click refresh. Make sure you are in the same AWS account (ID 176938214725).

Step 10: Test a restore (recommended)

```
aws s3 cp s3://backupbucket-176938214725-ap-south-1-an/mariadb/company_2026-09-21_05-03.sql.gz /tmp/
sudo mysql -e "CREATE DATABASE company_restore_test;"
gunzip < /tmp/company_2026-09-21_05-03.sql.gz | sudo mysql company_restore_test
sudo mysql -e "SELECT * FROM company_restore_test.employees;"
sudo mysql -e "DROP DATABASE company_restore_test;"
```

Downloads the backup, loads it into a temporary database, checks the rows are there, then deletes the test database. A backup is only proven when it restores.

Quick Troubleshooting

Problem	Fix
No match for argument: mariadb105-server	Check the OS with <code>cat /etc/os-release</code> and use the matching package name.
aws: command not found	Install AWS CLI v2 (Step 6).
Unable to locate credentials	Attach the IAM role to the EC2 instance.
AccessDenied	Attach an S3 policy to the role and use your real bucket name.
Unknown options (sync)	Remove spaces or line breaks in the <code>s3://</code> URL.
Folder not visible in console	Refresh, check the bucket name and account, and verify with <code>aws s3 ls</code> .

OUTPUT:

The screenshot shows the Amazon S3 console interface for the bucket `backupbucket-176938214725-ap-south-1-an`. The **Objects** tab is selected, showing a list of two objects:

Name	Type	Last modified	Size	Storage class
company_2026-09-21_05-03.sql.gz	gz	September 21, 2026, 11:06:49 (UTC+05:30)	770.0 B	Standard
mariadb/	Folder	-	-	-

The console also includes a search bar, a "Show versions" toggle, and various action buttons like "Copy S3 URI", "Copy URL", "Download", "Open", "Delete", "Actions", "Create folder", and "Upload".